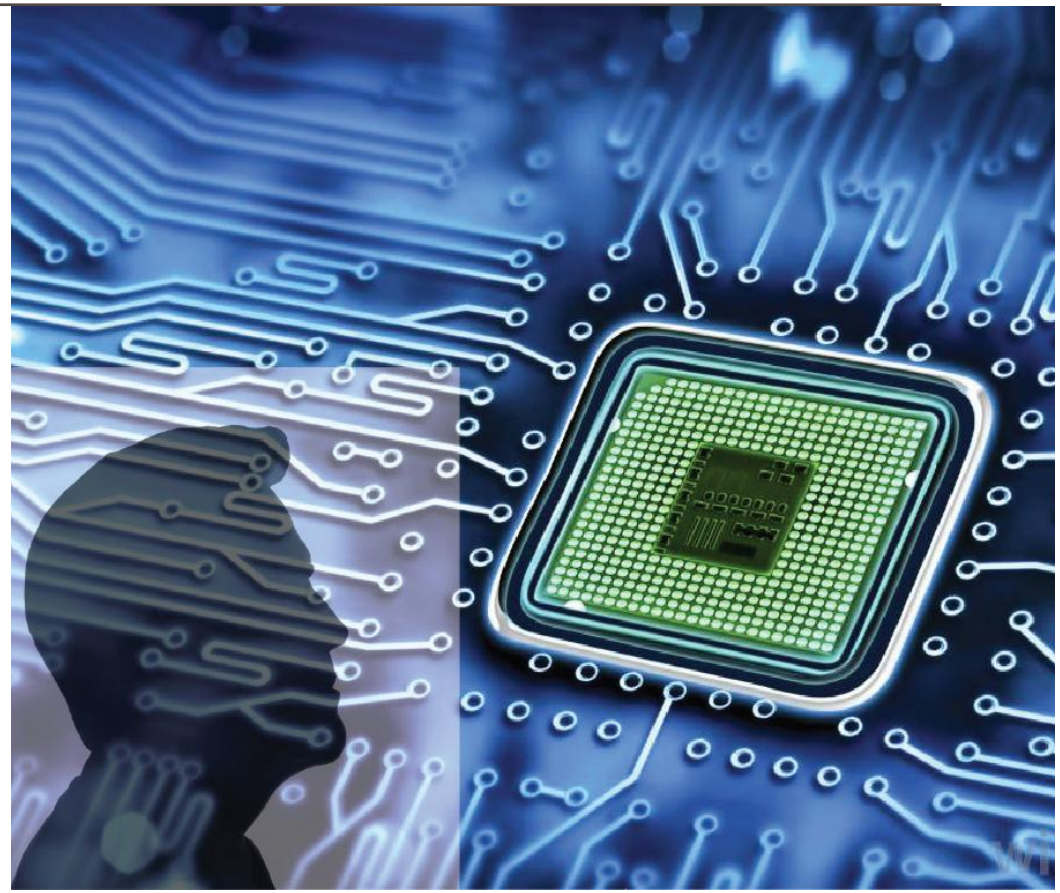
ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-11 Lecture-1
SystemVerilog



Sneh Saurabh
4th November 2025

Clarification: Weighted Distribution (1)

```
1 class Generator;
2   rand [0:1] mask_bits; // declare random number
3   constraint c1 { // define constraints
4     mask_bits dist {0:=1,1:=1,2:=1,3:=1};
5   }
6   function void display;
7     $display("Mask Bits: %0d", mask_bits);
8   endfunction
9 endclass

11 program MyRand();
12   initial begin
13     Generator gen;
14     gen = new(); // create object with random data
15     for (int i=0; i <10; i++) begin
16       gen.randomize(); // generate random values
17       gen.display();
18     end
19   end
20 endprogram
```

```
ncsim> run
Mask Bits: 1
Mask Bits: 1
Mask Bits: 2
Mask Bits: 0
Mask Bits: 1
Mask Bits: 0
Mask Bits: 3
Mask Bits: 2
Mask Bits: 2
Mask Bits: 2
```

Clarification: Weighted Distribution (2)

```
1 class Generator;
2   rand [0:1] mask_bits; // declare random number
3   constraint c1 { // define constraints
4     mask_bits dist {0:=1,3:=1};
5   }
6   function void display;
7     $display("Mask Bits: %0d", mask_bits);
8   endfunction
9 endclass
```

```
ncsim> run
Mask Bits: 3
Mask Bits: 3
Mask Bits: 0
Mask Bits: 0
Mask Bits: 3
Mask Bits: 0
Mask Bits: 3
Mask Bits: 0
Mask Bits: 0
Mask Bits: 0
```

Clarification: Weighted Distribution (3)

```
1 class Generator;
2   rand [0:1] mask_bits; // declare random number
3   constraint c1 { // define constraints
4     mask_bits dist {0:=1,3:=1.2};
5   }
6   function void display;
7     $display("Mask Bits: %0d", mask_bits);
8   endfunction
9 endclass
```

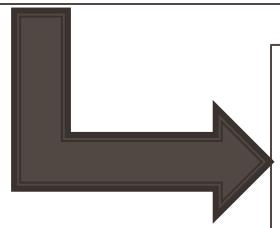
- Whether weight can be real numbers (besides integer)?

Language Reference Manual (LRM)

- The weight can be any **integral expression,** with the result of the expression being interpreted as an **unsigned value**.

```
ncsim: *W,RNDREJ: This constraint was not processed due to: unsupported floating point expression
                  mask_bits dist {0:=1,3:=1.2}; (./dist-3.sv,4)
```

SystemVerilog



Functional Coverage

Coverage Metrics

- Bins are the basic units of measurement for functional coverage
- Given a variable or an expression, SystemVerilog creates bins

Every time a cover group is triggered

- A “token” is dropped in the corresponding bin(s)
- At the end of simulation, a data base is created for all bins with “token”
- A separate analysis tool reads all databases and generates a report with the coverage for each part of the design and for the total coverage

Calculating coverage:

- **Hit count:** each bin has an associated counter that increments when the corresponding value is observed and hit count is incremented
- **Coverage of a bin:** A bin is considered "covered" if it meets or exceeds its hit threshold (default: 1).
- **Coverage for a cover point**: the ratio of the number of bins hit and the total number of bins
- **Coverage for a cover group:** Weighted average of coverage all the cover points (and cross coverage) in a group

Coverage: Automatic Bins (1)

- SystemVerilog automatically creates bins for cover points
  - ➢ Looks at the range of values for variable, expressions and based on it creates automatic bins
- We can restrict it using auto_bin_max (specifies the maximum number of bins that will be automatically created)

```
class Generator;
  rand [3:0]x; // declare random number
  rand [3:0]y; // declare random number
  covergroup MyCover;
    coverpoint x;
    coverpoint y {option.auto_bin_max = 2;}
  endgroup

  function new();
    MyCover = new();
  endfunction

   unction void display;
    $display("x: %0d y: %0d", x, y);
   ndfunction
   class
```

```
program MyRand();
  initial begin
    Generator gen;
    gen = new(); // create object with random data
    gen.MyCover.start();
    for (int i=0; i <10; i++) begin
      gen.randomize(); // generate random values
      gen.MyCover.sample();
      gen.display();
    end
    gen.MyCover.stop();
    $display("Coverage: %0d\%", gen.MyCover.get_coverage());
  end
endprogram
```

```
ncsim> run
x: 1 y: 13
x: 4 y: 10
x: 8 y: 5
x: 2 y: 11
x: 2 y: 2
x: 7 y: 2
x: 12 y: 3
x: 3 y: 10
x: 1 y: 7
x: 11 y: 12
Coverage:          75%
```

Coverage: Automatic Bins (2)

```
ncsim> run
x: 1 y: 13
x: 4 y: 10
x: 8 y: 5
x: 2 y: 11
x: 2 y: 2
x: 7 y: 2
x: 12 y: 3
x: 3 y: 10
x: 1 y: 7
x: 11 y: 12
Coverage:          75%
```

Bins x

Abstract Expand

Ex	UNR	Name	Overall Average Grade	Overall Covered	Score
		(no filter)	(no filter)	(no filter)	(no filter)
		auto[0]	0%	0 / 1 (0%)	0
		auto[1]	100%	1 / 1 (100%)	2
		auto[2]	100%	1 / 1 (100%)	2
		auto[3]	100%	1 / 1 (100%)	1
		auto[4]	100%	1 / 1 (100%)	1
		auto[5]	0%	0 / 1 (0%)	0
		auto[6]	0%	0 / 1 (0%)	0
		auto[7]	100%	1 / 1 (100%)	1
		auto[8]	100%	1 / 1 (100%)	1
		auto[9]	0%	0 / 1 (0%)	0
		auto[10]	0%	0 / 1 (0%)	0
		auto[11]	100%	1 / 1 (100%)	1
		auto[12]	100%	1 / 1 (100%)	1
		auto[13]	0%	0 / 1 (0%)	0
		auto[14]	0%	0 / 1 (0%)	0
		auto[15]	0%	0 / 1 (0%)	0

- Average of 50% and 100% is 75%

Bins y

Abstract Expand

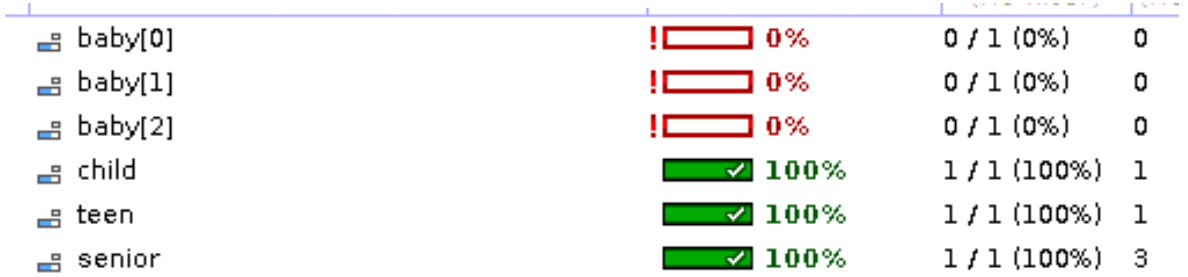
Ex	UNR	Name	Overall Average Grade	Overall Covered	Score
		(no filter)	(no filter)	(no filter)	(no filter)
		auto[0:7]	100%	1 / 1 (100%)	5
		auto[8:15]	100%	1 / 1 (100%)	5

Coverage: User-defined (1)

- We should use readable names for bins to ease report analysis
- Can create list of bins using []
  - Name chosen by the tool
- Can use dollar sign ($) on the left and right side of a range to specify that bins start/end at the lower and upper limit of the range
- Values not falling into user-defined bins goes to a default bin
  - default bin is not considered in coverage computation

```
class Generator;
  rand bit [6:0]age; // declare random number
  covergroup MyCover;
    infant: coverpoint age {
      bins baby[] = {[$:2]};
      bins child = {[3:12]};
      bins teen = {[12:19]};
      bins senior = {[65:$]};
    }
  endgroup

  function new();
    MyCover = new();
  endfunction

  function void display;
    $display("age: %0d", age);
  endfunction
endclass
```

```
ncsim> run
age: 109
age: 17
age: 106
age: 4
age: 85
Coverage:                50%
```



Coverage: Conditional

- Can add condition to a cover point using the keyword iff
  - ➢ Useful when we want to ignore hits under certain condition (example when reset is high)

- Only 2, 3, 5 considered as hit
- 7, 4 ignored

- Alternatively, we can use start() and stop() functions for cover groups

```
class Generator;
  rand bit reset;
  rand bit [2:0]data; // declare random number
  covergroup MyCover;
    coverpoint data iff (reset == 0);
  endgroup

  function new();
    MyCover = new();
  endfunction

  function void display;
    $display("reset: %b data: %0d", reset, data);
  endfunction
endclass
```

```
ncsim> run
reset: 1 data: 5
reset: 0 data: 2
reset: 0 data: 5
reset: 0 data: 3
reset: 0 data: 2
reset: 1 data: 2
reset: 0 data: 3
reset: 1 data: 2
reset: 1 data: 7
reset: 1 data: 4
Coverage:          38%
```

Coverage: Transition

- We can specify state transitions for a cover point
  - Using operator =>
- Allows us to trace the interesting sequences

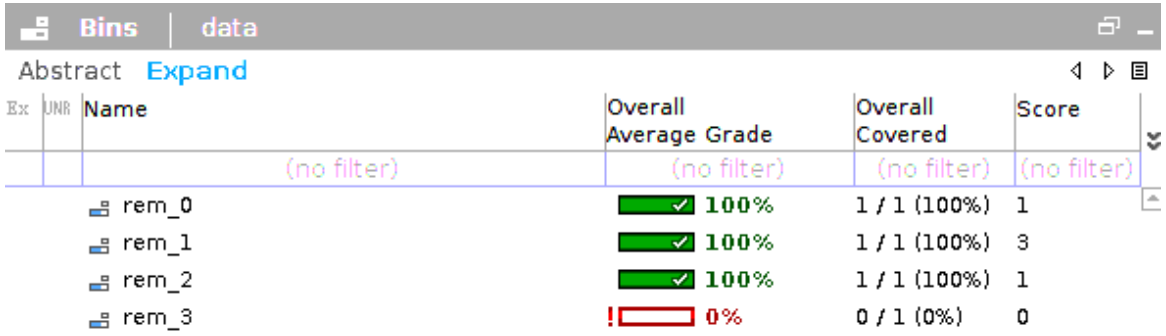
```
ncsim> run
state: 1
state: 1
state: 2
state: 0
state: 1
Coverage:          50%
```


```
class Generator;
  rand bit [1:0]state; // declare random number
  covergroup MyCover;
    coverpoint state {
      bins first = (0 => 1);
      bins second = (1 => 2);
      bins third = (2 => 3);
      bins fourth = (3 => 1);
    }
  endgroup

  function new();
    MyCover = new();
  endfunction

  function void display;
    $display("state: %0d", state);
  endfunction
endclass
```

Coverage: Wildcard

- Can use wildcard (any ? X Z) in the expression to match 0 or 1

```
ncsim> run
data: 237
data: 17
data: 234
data: 132
data: 85
Coverage:                75%
```




```
class Generator;
  rand bit [7:0]data; // declare random number
  covergroup MyCover;
    coverpoint data {
      wildcard bins rem_0 = {8'b??????00};
      wildcard bins rem_1 = {8'b??????01};
      wildcard bins rem_2 = {8'b??????10};
      wildcard bins rem_3 = {8'b??????11};
    }
  endgroup

  function new();
    MyCover = new();
  endfunction

  function void display;
    $display("data: %0d", data);
  endfunction
endclass
```

- Bins consist of remainders obtained if the number is divided by 4

Coverage: Ignoring Values and Illegal Bins

- For automatically created bins, we can use ignore_bins to specify the values to exclude from functional coverage calculation

```
ncsim> run
data: 5
data: 1
data: 2
data: 4
data: 5
data: 0
Coverage:          75%
```

Bins | data

Abstract Expand

Ex	UNR	Name	Overall Average Grade	Overall Covered	Score
		(no filter)	(no filter)	(no filter)	(no filter
		auto[0]	100%	1 / 1 (100%)	1
		auto[2]	100%	1 / 1 (100%)	1
		auto[4]	100%	1 / 1 (100%)	1
		auto[6]	0%	0 / 1 (0%)	0

```
class Generator;
  rand bit [2:0]data;
  covergroup MyCover;
    coverpoint data {
      ignore_bins odds = {1,3,5,7};
    }
  endgroup

  function new();
    MyCover = new();
  endfunction

  function void display;
    $display("data: %0d", data);
  endfunction
endclass
```

- If some sampled values should cause an error if observed, we can specify them using illegal_bins

```
ncsim: *E,EILLVS: (File: ./random-coverage-8.sv, Line: 5):(Time: 0 FS + 1) Sampled value
(5) for coverpoint (worklib.$unit_0x533ec9e1::Generator::MyCover@2_1.data) is an illegal
value.
```

Cross Coverage: Basics

- Sometimes we need to measure what values were seen by two or more cover points at the same time
- We use cross coverage metrics for this purpose
- If a variable has M values and another variable has N values then cross coverage would have $M \times N$ bins

- Cross coverage is defined using the keyword cross
- The cross construct records the combined values of two or more cover points in a group
- cross takes only cover points and simple variable names as inputs

```
class Generator;
  rand [1:0]rows; // declare random number
  rand [1:0]columns; // declare random number
  covergroup MyCover;
    coverpoint rows;
    coverpoint columns;
    cross rows, columns;
  endgroup

  function new();
    MyCover = new();
  endfunction

  function void display;
    $display("rows: %0d columns: %0d", rows, columns)
  endfunction
endclass
```

Cross Coverage: Example (1)

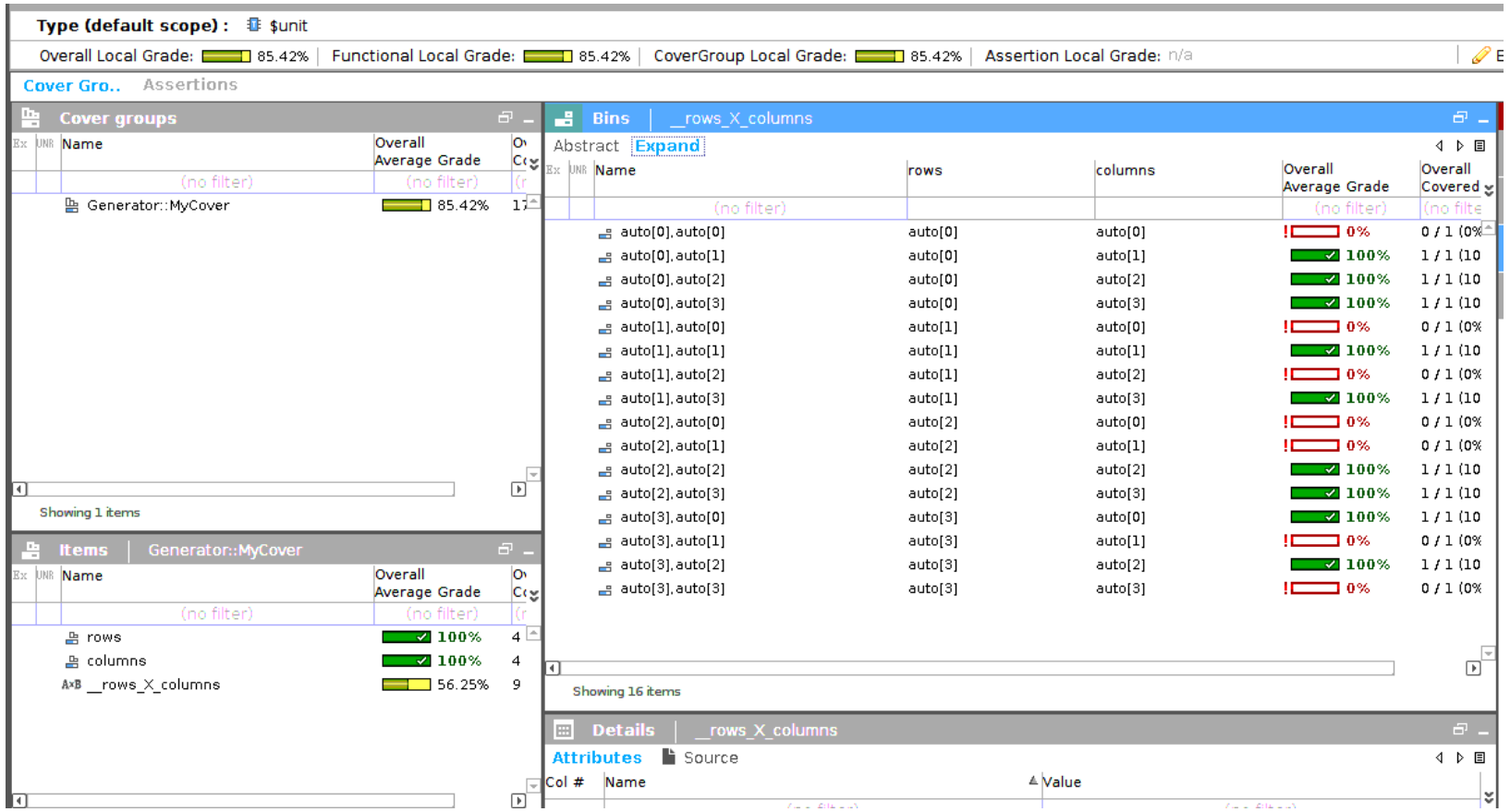
```
class Generator;
  rand [1:0]rows; // declare random number
  rand [1:0]columns; // declare random number
  covergroup MyCover;
    coverpoint rows;
    coverpoint columns;
    cross rows, columns;
  endgroup

  function new();
    MyCover = new();
  endfunction

  function void display;
    $display("rows: %0d columns: %0d", rows, columns)
  endfunction
endclass
```

```
program MyRand();
  initial begin
    Generator gen;
    gen = new(); // create object with random data
    gen.MyCover.start();
    for (int i=0; i <10; i++) begin
      gen.randomize(); // generate random values
      gen.MyCover.sample();
      gen.display();
    end
    gen.MyCover.stop();
    $display("Coverage: %0d\%", gen.MyCover.get_coverage());
  end
endprogram
```

```
ncsim> run
rows: 1 columns: 1
rows: 0 columns: 2
rows: 0 columns: 1
rows: 2 columns: 3
rows: 2 columns: 2
rows: 3 columns: 2
rows: 0 columns: 3
rows: 3 columns: 2
rows: 1 columns: 3
rows: 3 columns: 0
Coverage:          85%
```

Cross Coverage: Example (2)

```
ncsim> run
rows: 1 columns: 1
rows: 0 columns: 2
rows: 0 columns: 1
rows: 2 columns: 3
rows: 2 columns: 2
rows: 3 columns: 2
rows: 0 columns: 3
rows: 3 columns: 2
rows: 1 columns: 3
rows: 3 columns: 0
Coverage:                85%
```



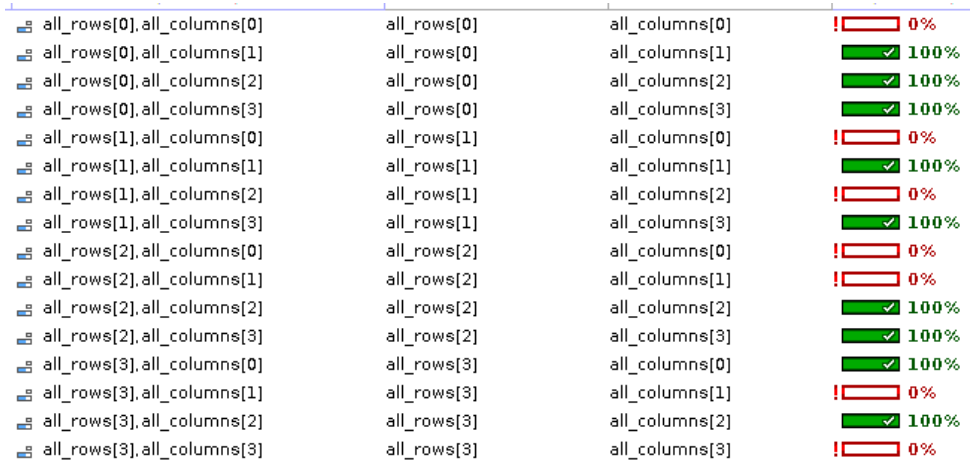
- Total reported coverage is the average of all bins (including cross cover bins)
- $\frac{100+100+56.25}{3} = 85.4$

Cross Coverage: User-defined Cover Points

- We can define cross coverage on user-defined cover points
- The names will be easy to understand in the report of cross coverage

- We can define weights to various cover points using type_option.weight

```
ncsim> run
rows: 1 columns: 1
rows: 0 columns: 2
rows: 0 columns: 1
rows: 2 columns: 3
rows: 2 columns: 2
rows: 3 columns: 2
rows: 0 columns: 3
rows: 3 columns: 2
rows: 1 columns: 3
rows: 3 columns: 0
Coverage:              56%
```




```
class Generator;
  rand bit [1:0]rows; // declare random number
  rand bit [1:0]columns; // declare random number
  covergroup MyCover;
    myrows: coverpoint rows {
      bins all_rows[] = {[0:$]};
      type_option.weight=0;}

    mycolumns: coverpoint columns {
      bins all_columns[] = {[0:$]};
      type_option.weight=0;}

   rows_with_columns: cross myrows, mycolumns;
  endgroup

  function new();
    MyCover = new();
  endfunction

  function void display;
    $display("rows: %0d columns: %0d", rows, columns);
  endfunction
endclass
```

Cross Coverage: Subset (1)

- We can use keyword ignore_bins to exclude some cross coverage bins
- We use the keyword intersect to define a subset of values in the bins

- Bins considered:
  - (0,1), (0,2)
  - (1,1), (1,2)
  - (2,1), (2,2)
  - (3,1), (3,2)

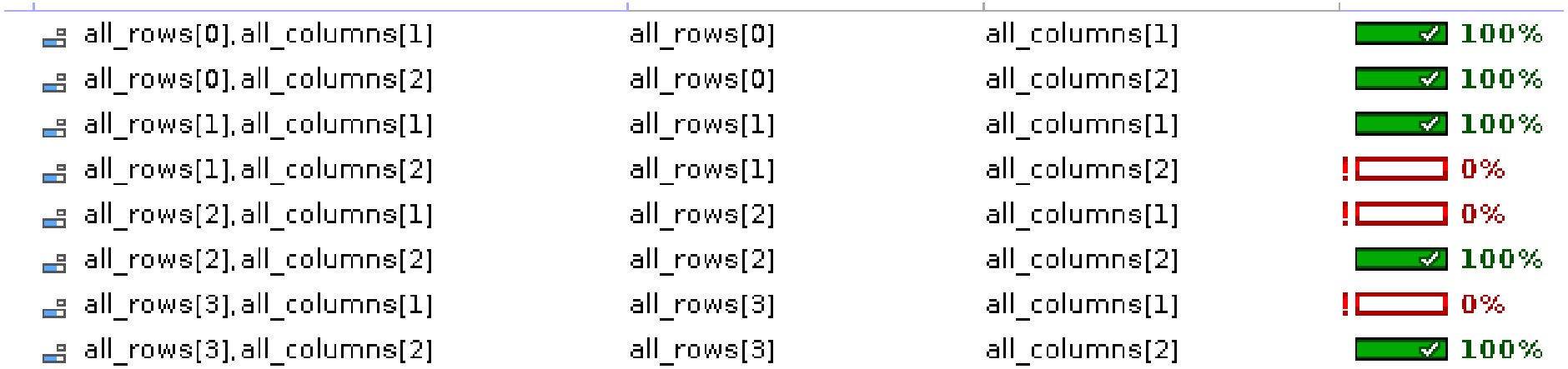
```
class Generator;
  rand bit [1:0]rows; // declare random number
  rand bit [1:0]columns; // declare random number
  covergroup MyCover;
    myrows: coverpoint rows {
      bins all_rows[] = {[0:$]}; type_option.weight=0;}
    mycolumns: coverpoint columns {
      bins all_columns[] = {[0:$]}; type_option.weight=0;}
    rows_with_columns: cross myrows, mycolumns {
      ignore_bins rows_with_column_0 = binsof(myrows.all_rows) &&
        binsof(mycolumns.all_columns) intersect {0};
      ignore_bins rows_with_column_3 = binsof(myrows.all_rows) &&
        binsof(mycolumns.all_columns) intersect {3};
    }
  endgroup

  function new();
    MyCover = new();
  endfunction

  function void display;
    $display("rows: %0d columns: %0d", rows, columns);
  endfunction
endclass
```

Cross Coverage: Subset (2)

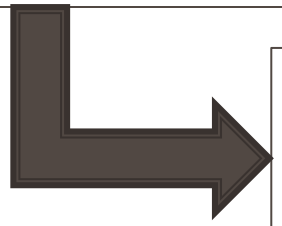
```
ncsim> run
rows: 1 columns: 1
rows: 0 columns: 2
rows: 0 columns: 1
rows: 2 columns: 3
rows: 2 columns: 2
rows: 3 columns: 2
rows: 0 columns: 3
rows: 3 columns: 2
rows: 1 columns: 3
rows: 3 columns: 0
Coverage:          63%
```

- Bins considered:
  - (0,1), (0,2)
  - (1,1), (1,2)
  - (2,1), (2,2)
  - (3,1), (3,2)

- Sometimes we are interested in only a subset of automatically created cross coverage bins
  - We can define them explicitly in the cross cover point



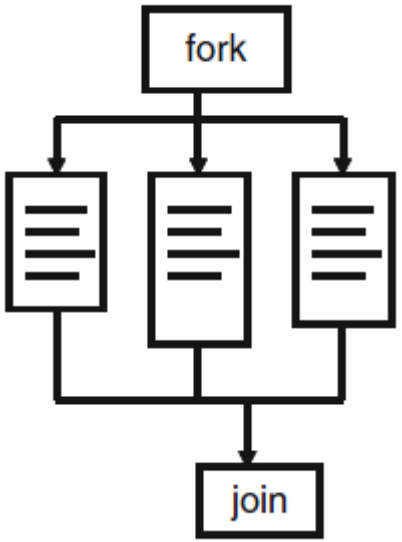
SystemVerilog



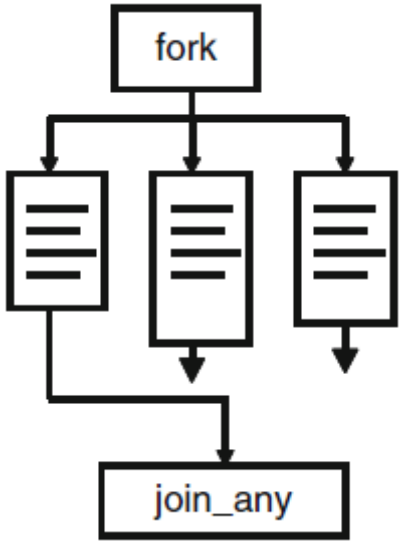
Threads

SystemVerilog Threads: Basics

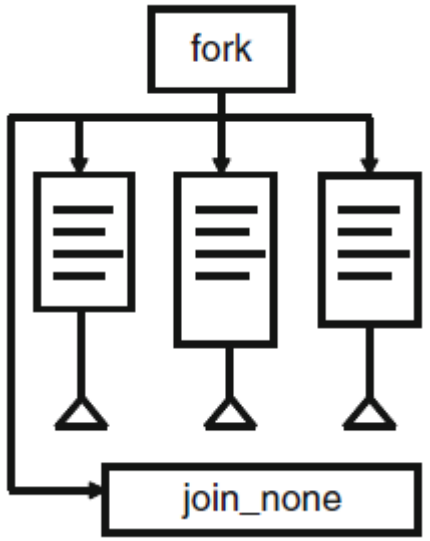
Three types
- fork … join
- fork … join_any
- fork … join_none



All statements within fork-join should finish before rest of the code can execute



If any statement within fork-join finishes, rest of the code can execute



Does not wait for any statement within fork-to finishes before moving to the code can execute

fork … join

```
program TestFork();
  initial begin
    $display("@%0t: start", $time);
    #10 $display("@%0t: sequential delay", $time);
    fork
      #30 $display("@%0t: fork 1", $time);
      #10 $display("@%0t: fork 2", $time);
      #20 $display("@%0t: fork 3", $time);
    join
    $display("@%0t: joined now ...", $time);
  end
endprogram
```

```
ncsim> run
@0: start
@10: sequential delay
@20: fork 2
@30: fork 3
@40: fork 1
@40: joined now ...
```

fork … join_any

```
program TestFork();
  initial begin
    $display("@%0t: start", $time);
    #10 $display("@%0t: sequential delay", $time);
    fork
      #30 $display("@%0t: fork 1", $time);
      #10 $display("@%0t: fork 2", $time);
      #20 $display("@%0t: fork 3", $time);
    join_any
    $display("@%0t: joined now ...", $time);
    #100;
  end
endprogram
```

```
ncsim> run
@0: start
@10: sequential delay
@20: fork 2
@20: joined now ...
@30: fork 3
@40: fork 1
```

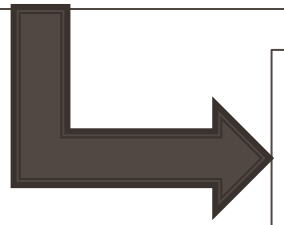
- If there is no #100, $finish will be called immediately after join (other threads will not have opportunity to display)

fork … join_none

```
program TestFork();
  initial begin
    $display("@%0t: start", $time);
    #10 $display("@%0t: sequential delay", $time);
    fork
      #30 $display("@%0t: fork 1", $time);
      #10 $display("@%0t: fork 2", $time);
      #20 $display("@%0t: fork 3", $time);
    join_none
    $display("@%0t: joined now ...", $time);
    #100;
  end
endprogram
```

```
ncsim> run
@0: start
@10: sequential delay
@10: joined now ...
@20: fork 2
@30: fork 3
@40: fork 1
```

SystemVerilog



Interprocess Communication

Inter Process Communication: Basics

- Typically, a testbench spawns multiple threads
- These threads need to synchronize, exchange data, and access resources
  - Example: Multiple threads can try to access the bus of a DUT
  - Testbench needs to ensure that only one thread is granted access
- These operations needs Inter Process communication (IPC)

Three constructs are provided in SystemVerilog for IPC

- Event
- Semaphore
- Mailboxes

IPC: Event (1)

- Event is declared using the event keyword.
- Triggered using the -> operator
- Wait for an event to occur using the @ operator

- Event is a handle to a synchronization object in SystemVerilog
- Allows event to be passed as objects and shared

```
program TestFork();
  initial begin
    event e;
    fork
      begin
        @e;
        $display("Event e triggered, and observed in first thread at %0t", $time);
      end

      begin
        @e;
        $display("Event e triggered, and observed in second thread at %0t", $time);
      end

      begin
        #10 $display("Trigerring event e at: %0t", $time);
        -> e;
      end
    join
  end
endprogram
```

```
Trigerring event e at: 10
Event e triggered, and observed in first thread at 10
Event e triggered, and observed in second thread at 10
```

IPC: Event (2)

Synchronization issue in @ and ->:

- The waiting process must execute the @ statement before the triggering process executes the trigger operator ->
  - Else the waiting process remains blocked

- If the event triggering (->) and waiting (@) happens at the same time slot, waiting process may miss detecting the event trigger

- To avoid above problem: wait(event_name.triggered);
- A process that waits on the triggered state always unblocks if that event is triggered in the given time slot (regardless of the order of wait and trigger)

IPC: Semaphore

- Can create a semaphore object with one or more keys using new
- Get a key using:
  - get(): blocking
  - try_get(): non-blocking
- Release the key: put()
- Similar to SystemC

IPC: Mailboxes (1)

- Mechanism to transfer data between processes
- Acts like a FIFO of SystemC

- Declared using: mailbox #(data_type)
- Can be of fixed size or can be unbounded (default)

Methods

- put(): Adds an item to the mailbox (blocks if full for bounded mailboxes).
- get(): Removes and retrieves an item from the mailbox (blocks if empty).
- try_put(): Non-blocking version of put().
- try_get(): Non-blocking version of get().
- num(): current number of items

IPC: Mailboxes

```
class Transaction;
  bit [7:0]data;
endclass
```

```
program TestFork();
  initial begin
    mailbox #(Transaction) mbox;
    Transaction tr_sent;
    Transaction tr_received;
    mbox = new();
    fork
      begin
        tr_sent = new();
        $display("Sender process start at: %0t", $time);
        #10;
        tr_sent.data = 25;
        $display("Sending transaction at: %0t data: %0d", $time, tr_sent.data);
        mbox.put(tr_sent);
        #20;
        tr_sent.data = 45;
        $display("Sending transaction at: %0t data: %0d", $time, tr_sent.data);
        mbox.put(tr_sent);
      end

      begin
        $display("Receiever process start at: %0t", $time);
        mbox.get(tr_received);
        $display("Received transaction at: %0t data: %0d", $time, tr_received.data);
        mbox.get(tr_received);
        $display("Received transaction at: %0t data: %0d", $time, tr_received.data);
      end
    join
  end
endprogram
```

```
Sender process start at: 0
Receiever process start at: 0
Sending transaction at: 10 data: 25
Received transaction at: 10 data: 25
Sending transaction at: 30 data: 45
Received transaction at: 30 data: 45
```

References

- Spear, Chris. SystemVerilog for Verification: A Guide to Learning the Testbench Language Features. Vol. 161. Springer, 2008.